SKILL VAULT: STUDENT PORTFOLIO MANAGEMENT SYSTEM

MINOR PROJECT REPORT

By

ABHISHEK NEGI (RA2511056030023)

AJAY RAJ CHAUHAN (RA2511056030021)

SARTHAK NAURIYAL (RA2511056030007)

Under the guidance of

Ms. Neha

In partial fulfilment for the Course

of

21CSC206P – ADVANCED OBJECT ORIENTED & PROGRAMMING

in

Department of Computer Science & Engineering



FACULTY OF ENGINEERING AND TECHNOLOGY

SCHOOL OF COMPUTING

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

DELHI - NCR CAMPUS

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this minor project report for the course **21CSC206P ADVANCED OBJECT ORIENTED & PROGRAMMING** entitled in "**Skill Vault: Student Portfolio Management System**" is the bonafide work of **Abhishek Negi (RA2511056030023)**, **Ajay Raj Chauhan (RA2511056030021)** and **Sarthak Nauriyal (RA2511056030007)** who carried out the work under my supervision.

SIGNATURE

Ms. Neha
Assistant Professor
Computer Science & Engineering

SIGNATURE

Dr. Neelam Singh
Head of the department
Computer Science & Engineering

ABSTRACT

Students build their profiles through coursework, self-learning, certifications and projects, but this evidence is usually scattered across notebooks, cloud folders, learning platforms and code repositories. Gathering it for an internship or placement application is slow and error-prone.

Skill Vault is a Java-based desktop application that gives students one structured place to maintain this record. A student can register and log in, record skills with proficiency levels, manage project details with technologies and GitHub links, store certificate information including issuer, completion date and URL, and view everything together in a consolidated portfolio.

The application uses Java Swing for the graphical user interface, JDBC for communication with the database, and MySQL for persistent storage. The code is organised into user-interface, model, data-access (DAO) and database-connection components, which keeps responsibilities separate and the code easy to maintain. This report describes the requirements, architecture, database design, implementation, working screenshots, test cases and future scope of the system.

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our guide, Ms. Neha, for her valuable guidance, consistent encouragement, personal caring, timely help and for providing us with an excellent atmosphere for doing this project. All through the work, in spite of her busy schedule, she has extended cheerful and cordial support to us for completing this project work.

Abhishek Negi (RA2511056030023)

Ajay Raj Chauhan (RA2511056030021)

Sarthak Nauriyal (RA2511056030007)

TABLE OF CONTENTS

CHAPTER NO	CONTENTS	PAGE NO
1	**INTRODUCTION**	1
	1.1 Motivation	1
	1.2 Objective	1
	1.3 Problem Statement	1
	1.4 Challenges	1
2	**EXISTING PROBLEMS**	3
3	**REQUIREMENT ANALYSIS**	4
	3.1 Requirement Analysis	4
	3.2 Hardware Requirement	4
	3.3 Software Requirement	4
	3.4 Input Data and Expected Outputs	5
4	**ARCHITECTURE & DESIGN**	6
	4.1 System Architecture	6
	4.2 System Workflow	6
	4.3 Database Schema Design	6
	4.4 Object-Oriented Design Considerations	7
5	**IMPLEMENTATION**	8
	5.1 Technologies Used	8
	5.2 Program Organisation	8
	5.3 Database Operations	9
	5.4 Validation and Error Handling	10
6	**EXPERIMENT RESULTS & ANALYSIS**	11
	6.1 Login and Registration	11
	6.2 Dashboard	12
	6.3 Skills Management	13
	6.4 Projects Management	14
	6.5 Certificates Management	16
	6.6 Student Portfolio	17
	6.7 Software Testing	18
	6.8 Limitations and Known Issues	19
7	**CONCLUSION**	20
	7.1 Conclusion	20
	7.2 Future Scope	20
8	**REFERENCES**	21

1. INTRODUCTION

Students build their skills through coursework, personal learning, workshops, certifications and projects. These achievements are often scattered across notebooks, documents, learning platforms and code repositories. Keeping the information organised, and turning it into a portfolio when an opportunity appears, becomes time-consuming.

Skill Vault is a desktop application that provides a single place to maintain this information. A registered student can sign in, save technical skills with a proficiency level, maintain project descriptions and GitHub links, record certificates with issuer and completion date, and view a consolidated student portfolio. The application is developed in Java, using Swing for the interface and MySQL for data persistence.

1.1 Motivation

Preparing for internships, placements or academic reviews requires a student to collect skills, project links and certificates from many places. A simple desktop tool that keeps these records together, and can present them as one portfolio, saves time and reduces the chance of missing an achievement. The project also gives the team a practical way to apply object-oriented programming, GUI design and database connectivity in one application.

1.2 Objective

1. To provide student registration and login screens.
2. To allow students to add, view, update and delete skills and projects.
3. To store certificate name, issuer, completion date and certificate URL.
4. To display student information and saved achievements in a consolidated portfolio.
5. To demonstrate object-oriented design, Java Swing GUI programming, and database connectivity using JDBC and MySQL.

1.3 Problem Statement

There is no simple, unified desktop tool that lets a student organise skills, projects and certificates in one place. Skill Vault addresses this by providing a structured interface and database-backed storage, so that each student can manage their own records and view them together as a portfolio.

1.4 Challenges

- **Design:** Separating the interface, model and data-access code so that each part can be understood and changed independently.
- **Data consistency:** Keeping the screens consistent with the data stored in MySQL after every add, update and delete.
- **Validation:** Validating inputs such as required fields, the semester range and dates before saving.
- **Security:** Handling database credentials and passwords safely, which needs hashing and external configuration in a production version.
- **Display accuracy:** Showing the same data correctly in every view; testing revealed a display issue in the skills list (see Section 6.8).

2. EXISTING PROBLEMS

Students typically keep skill lists, project information, certificate links and personal academic details in separate files or online services. This causes several recurring difficulties:

1. Information is fragmented, so no single, consistent profile exists.
2. Regular updates are easy to forget and hard to track across different tools.
3. Preparing for internships, placements or academic reviews requires gathering achievements manually each time.
4. Certificates, project links and skill levels are not linked to one student record, so presenting them together takes extra effort.

The problem addressed by Skill Vault is the absence of a simple, unified desktop tool for organising these records in a structured way.

3. REQUIREMENT ANALYSIS

3.1 Requirement Analysis

From the given scenario, we draw the following requirements:

1. Registration: capture full name, email, password, college, branch and semester for a new student.
2. Login: check the submitted email and password and open the dashboard after successful authentication.
3. Manage Skills: save a skill with a proficiency level; view saved skills; update or delete a selected skill.
4. Manage Projects: save a project title, description, technologies used and GitHub URL; update or delete project entries.
5. Manage Certificates: record certificate name, issuer, completion date and certificate URL; view saved certificates.
6. My Portfolio: present student information, skills, certificates and projects in a consolidated view.
7. Persistence and data isolation: store data in MySQL and link every record to the student account through student_id.
8. Maintainability: keep interface, model, data-access and connection code in separate packages.

3.2 Hardware Requirement

A standard computer capable of running a supported Java Development Kit and MySQL Server is sufficient. The suggested baseline is:

- Processor: Intel Core i3 or equivalent
- Memory: 4 GB RAM or more
- Storage: adequate disk space for the JDK, IDE, MySQL and project files

These are practical recommendations rather than measured minimum requirements.

3.3 Software Requirement

Table 3.1: Software requirements

Component	**Requirement**
Programming language	Java (JDK)
GUI library	Java Swing
Database	MySQL Server
Database connectivity	JDBC with MySQL Connector/J
Development environment	Visual Studio Code

3.4 Input Data and Expected Outputs

Inputs: student details and login credentials; skill names and proficiency levels; project title, description, technologies and repository URL; certificate name, issuer, completion date and URL.

Outputs: confirmation or validation messages, updated lists of saved records, and the generated student portfolio.

4. ARCHITECTURE & DESIGN

4.1 System Architecture

Skill Vault follows a layered structure. The Swing user-interface classes collect user input and display results. Model classes represent Student, Skill, Project and Certificate data. DAO (Data Access Object) classes execute database operations, and DBConnection provides the JDBC connection to the MySQL database.

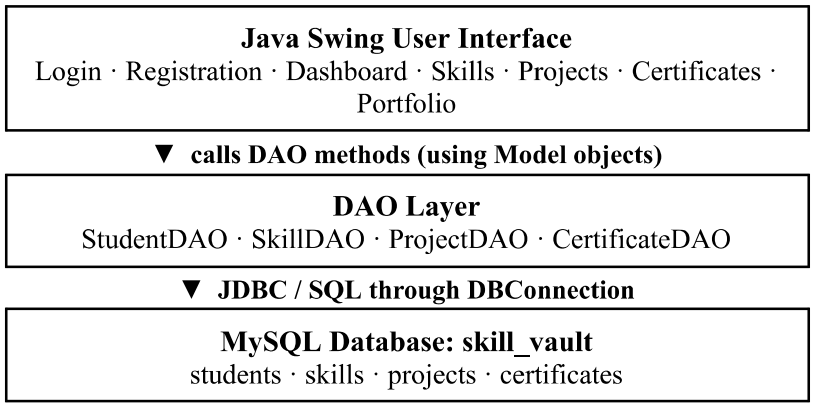


Figure 4.1: System architecture of Skill Vault

4.2 System Workflow

1. The student registers with personal and academic details.
2. The student logs in using the registered email and password.
3. The dashboard opens and provides navigation to the application modules.
4. The selected module uses its DAO class to insert, retrieve, update or delete records in MySQL.
5. The portfolio view retrieves the saved profile data and displays the student's skills, certificates and projects.

4.3 Database Schema Design

The database script creates four tables. The students table is the parent; skills, projects and certificates are linked to a student through student_id. The foreign keys use ON DELETE CASCADE, so deleting a student also removes the associated child records. The student email is declared unique and required.

Table 4.1: Database tables

Table	Key fields	Purpose
students	student_id (PK), name, email, password, college, branch, semester	Account and academic profile
skills	skill_id (PK), student_id (FK), skill_name, proficiency	Skills and proficiency
projects	project_id (PK), student_id (FK), title, description, technologies, github_url	Project details
certificates	certificate_id (PK), student_id (FK), title, issuer, completion_date, certificate_url	Certificate records

4.4 Object-Oriented Design Considerations

The project applies the object-oriented ideas taught in Advanced Object Oriented Programming:

- **Encapsulation:** Model classes represent real-world entities (Student, Skill, Project, Certificate) and keep their data together.
- **Abstraction and separation of concerns:** The UI calls DAO methods and does not deal with SQL directly, so database details are hidden from the interface.
- **Modularity:** Each module has its own DAO and its own screen, so one module can change without disturbing the others.

5. IMPLEMENTATION

5.1 Technologies Used

Java is used for application logic and class design. Java Swing provides windows, labels, text fields, password fields, buttons, lists and scroll panes. JDBC connects the Java program to MySQL, and SQL statements are executed by the DAO classes to store and retrieve information. The project archive includes the MySQL Connector/J library and a database.sql script.

5.2 Program Organisation

Table 5.1: Main source-code components

Package / file group	**Responsibility**
app.Main	Application entry point; launches the login interface.
ui.*	Swing screens for login, registration, dashboard, skills, projects, certificates and portfolio.
model.*	Data classes for Student, Skill, Project and Certificate.
dao.*	Database operations for student accounts, skills, projects and certificates.
database.DBConnection	Creates the JDBC connection to the skill_vault database.

The project follows a standard Java source layout. The screenshot below shows the actual project structure in the editor: the app, dao, database, model and ui packages under src/main/java, the MySQL Connector/J library in the lib folder, and the database.sql script.



Figure 5.1: Project structure of Skill Vault

5.3 Database Operations

The DAO classes encapsulate data access. The user-interface classes call the relevant DAO methods after a button is pressed. Primary keys identify each record uniquely, and foreign keys connect each skill, project and certificate to a student.

All DAO classes use parameterised PreparedStatement queries with placeholders (?) for every value, which keeps SQL separate from user input. The figure below shows the source code of the four DAO classes: StudentDAO (register a student), SkillDAO (save, list, update and delete skills), ProjectDAO (save and list projects) and CertificateDAO (save and list certificates).

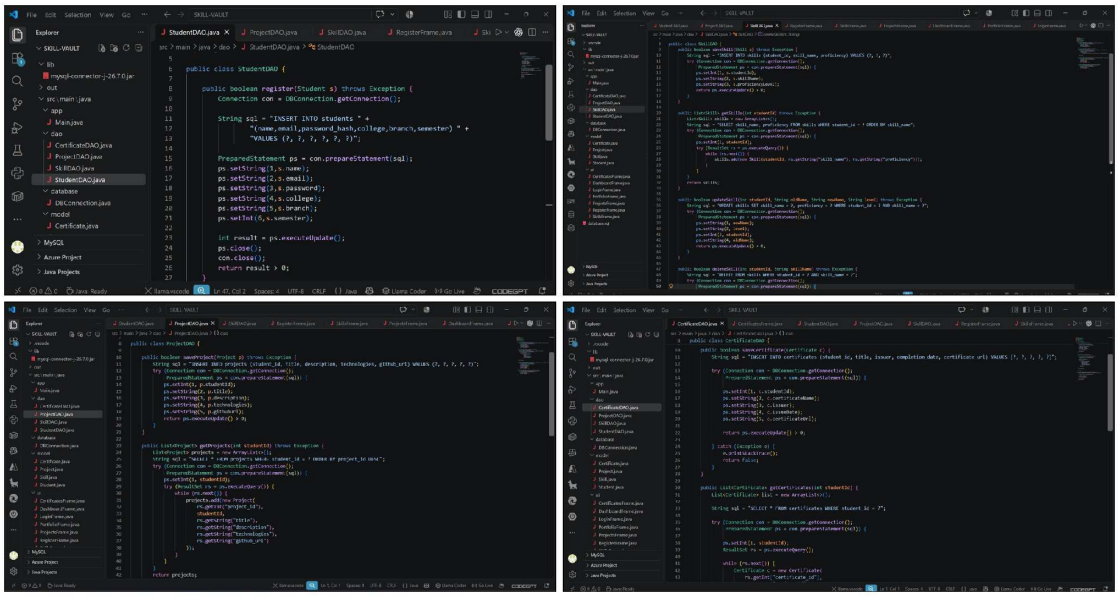


Figure 5.2: DAO classes – StudentDAO, SkillDAO, ProjectDAO and CertificateDAO

5.4 Validation and Error Handling

The registration screen checks that the required fields are filled and that the semester is numeric and within the accepted range of 1 to 12. The login screen shows a message for invalid credentials or connection failures. The module screens show status messages when records are loaded or actions are performed. Further validation for email format, date format, duplicate records and URL correctness can be added.

For example, when an email or password that does not match a registered account is entered on the login screen, the application stays on the login window and displays the message “Invalid email or password”, as shown below.

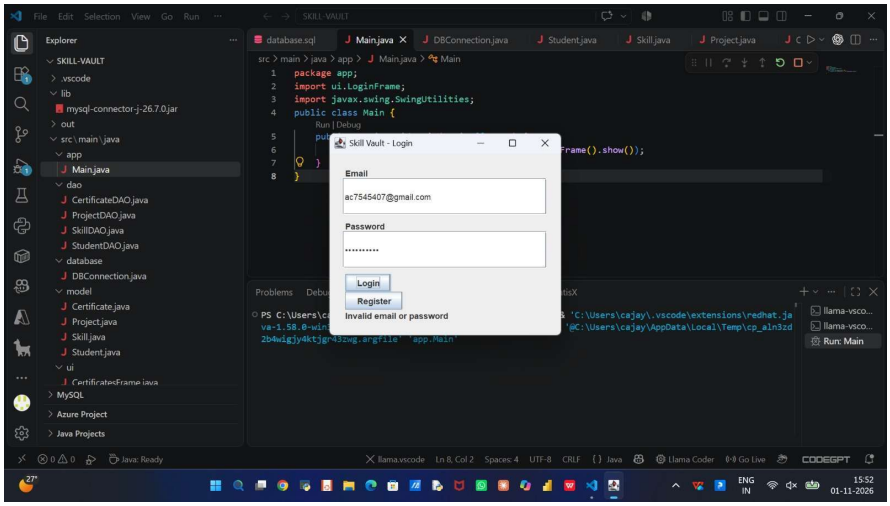


Figure 5.3: Login error message for invalid credentials

6. EXPERIMENT RESULTS & ANALYSIS

This chapter presents the working screenshots of the application and the test cases used to check it. Each figure shows an actual screen of the submitted project.

6.1 Login and Registration

The login screen accepts the student's email and password. Login attempts authentication; Register opens the registration form for a new user.

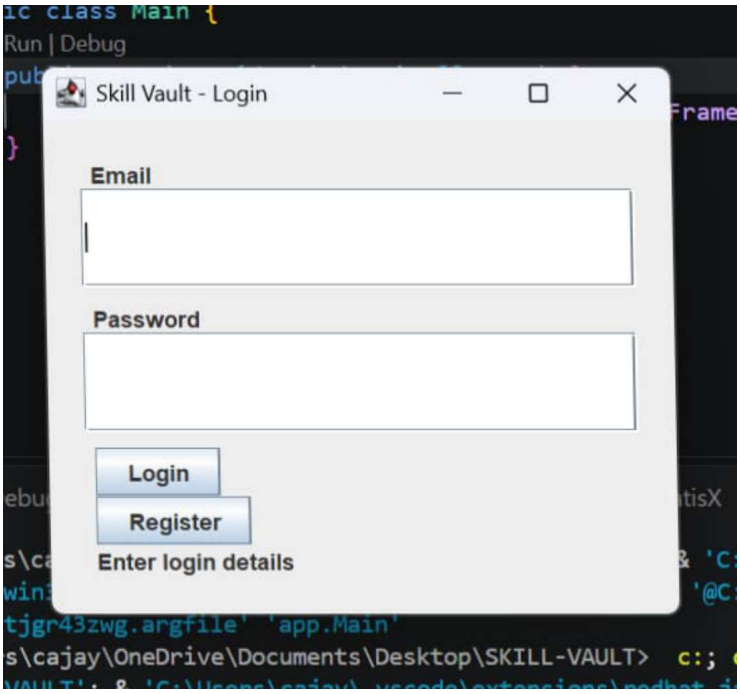


Figure 6.1: Login screen

The registration screen collects full name, email, password, college, branch and semester, and provides a route back to the login screen.

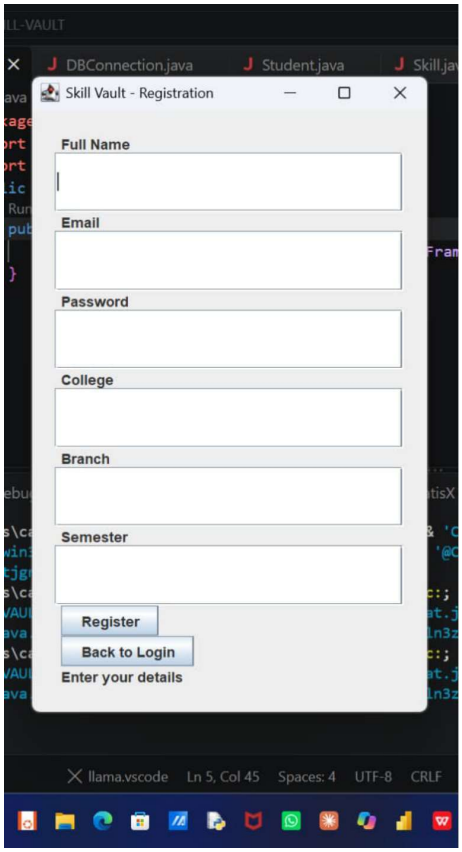


Figure 6.2: Registration screen

6.2 Dashboard

After login, the dashboard offers Manage Skills, Manage Projects, Manage Certificates, My Portfolio and Logout.

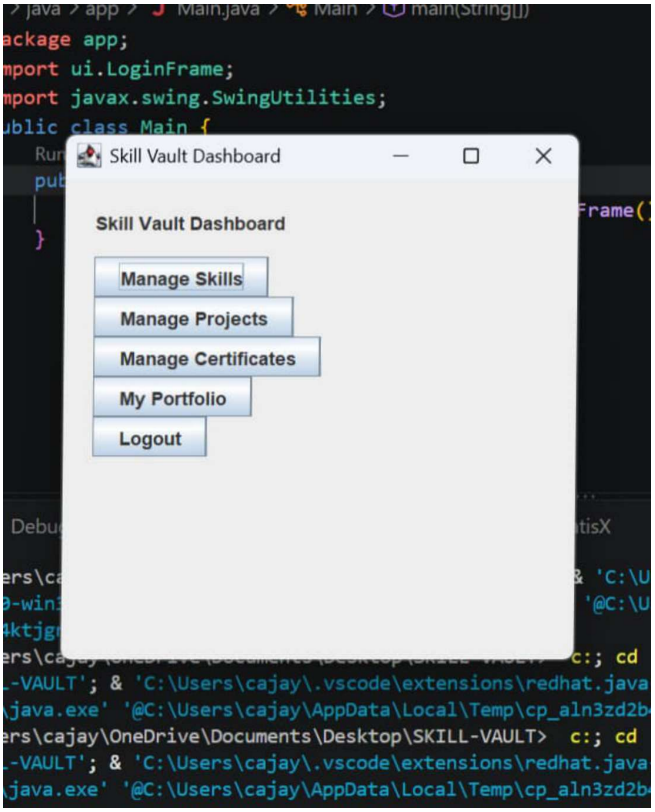


Figure 6.3: Main dashboard

6.3 Skills Management

The skills screen accepts a skill name and a proficiency level (Beginner is selected by default). Saved skills appear in a list, and buttons allow the student to add, update, delete and refresh records. In this screenshot the list entries show “null” in place of the proficiency level; this display issue is recorded in Section 6.8. The portfolio view (Figure 6.8) shows the same skills with their proficiency levels correctly.

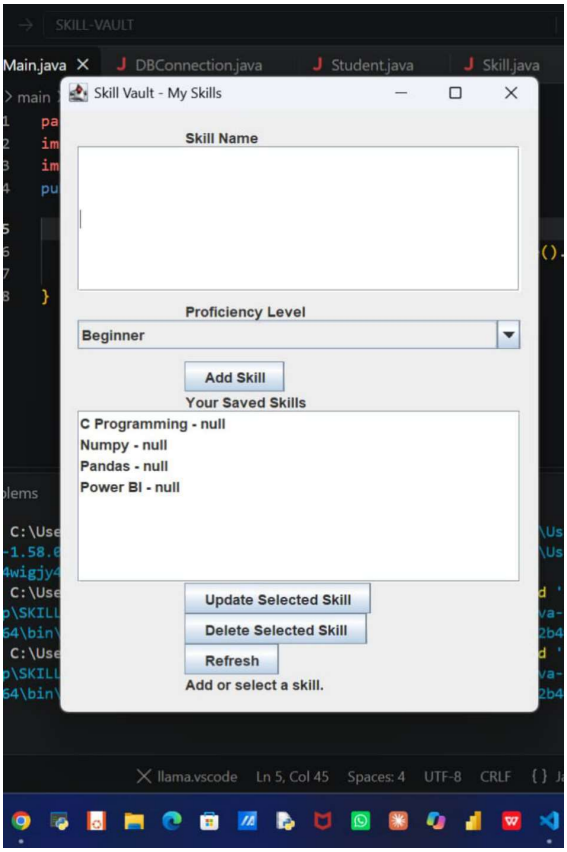


Figure 6.4: Skills management screen

6.4 Projects Management

The project screen has fields for project title, description, technologies used and GitHub URL, with buttons to add, update, delete, refresh and clear.

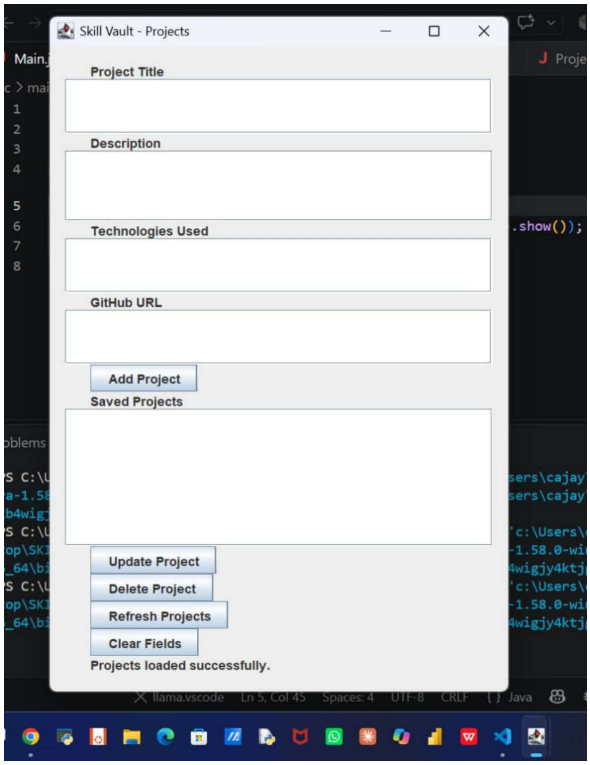


Figure 6.5: Projects management screen (empty form)

A saved project (Event_Countdown_Timer) is selected from the list and its details are loaded into the form so that they can be edited and saved with Update Project.

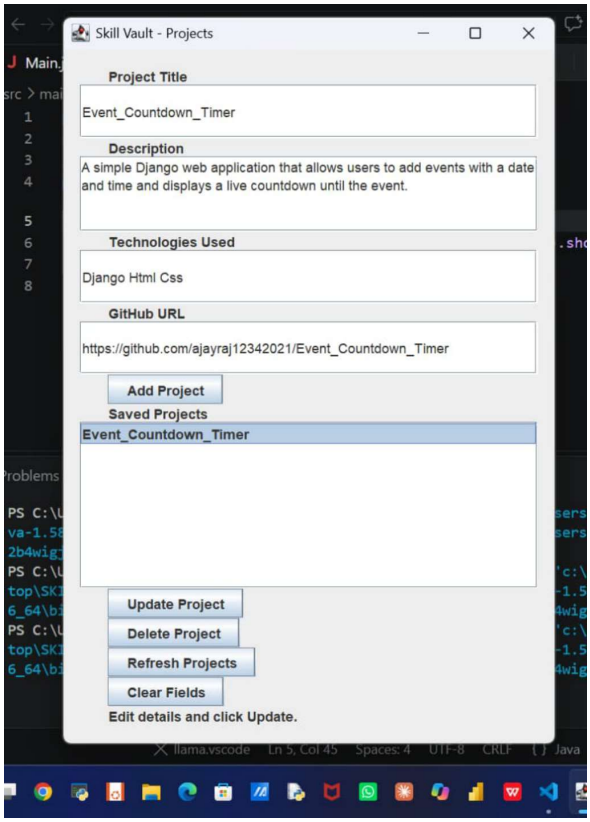


Figure 6.6: Project saved and selected for update

6.5 Certificates Management

The certificate screen records certificate name, issuer, completion date (YYYY-MM-DD) and certificate URL, and lists the saved certificates, such as the C++ Course Certificate shown.

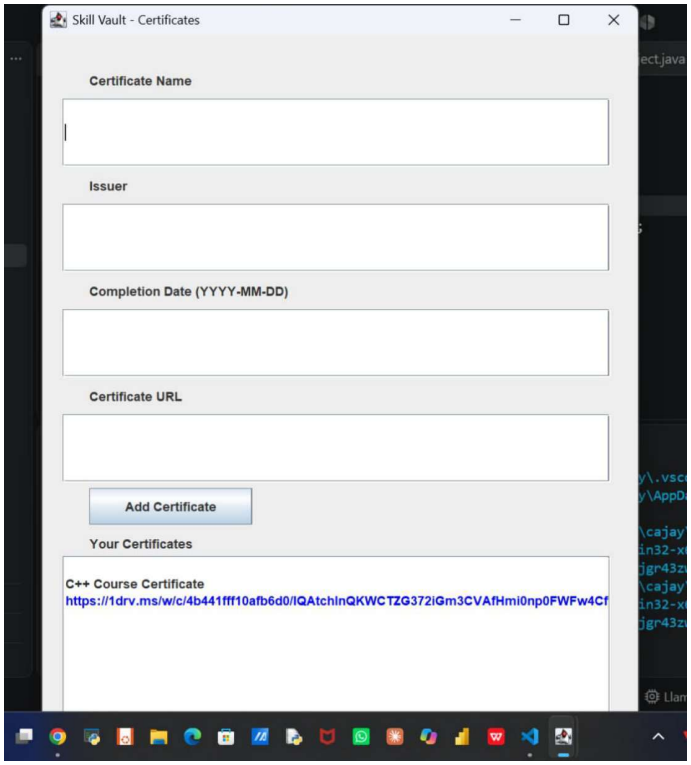


Figure 6.7: Certificates management screen

6.6 Student Portfolio

The portfolio consolidates personal information, skills with proficiency, certificates and projects in one scrollable view.

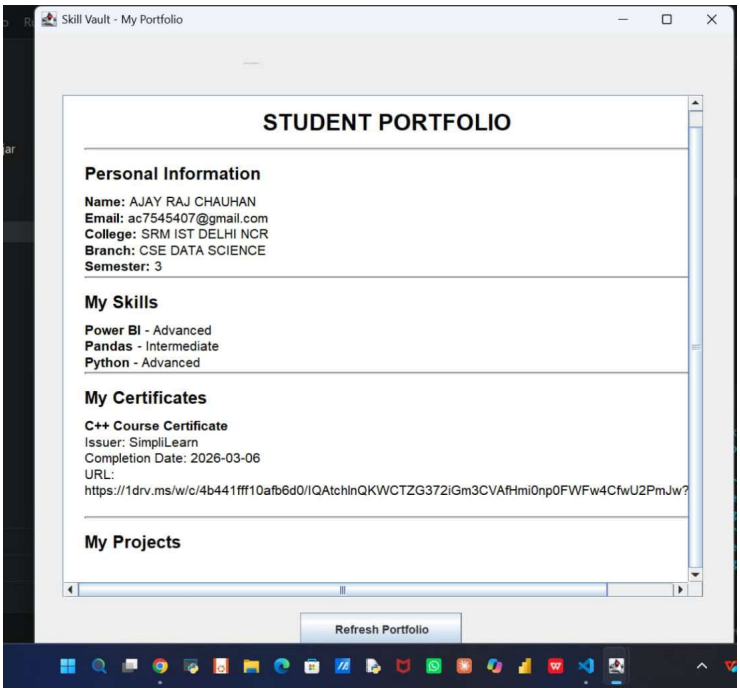


Figure 6.8: Student portfolio view

After scrolling, the portfolio shows each project with its description, technologies and GitHub link.

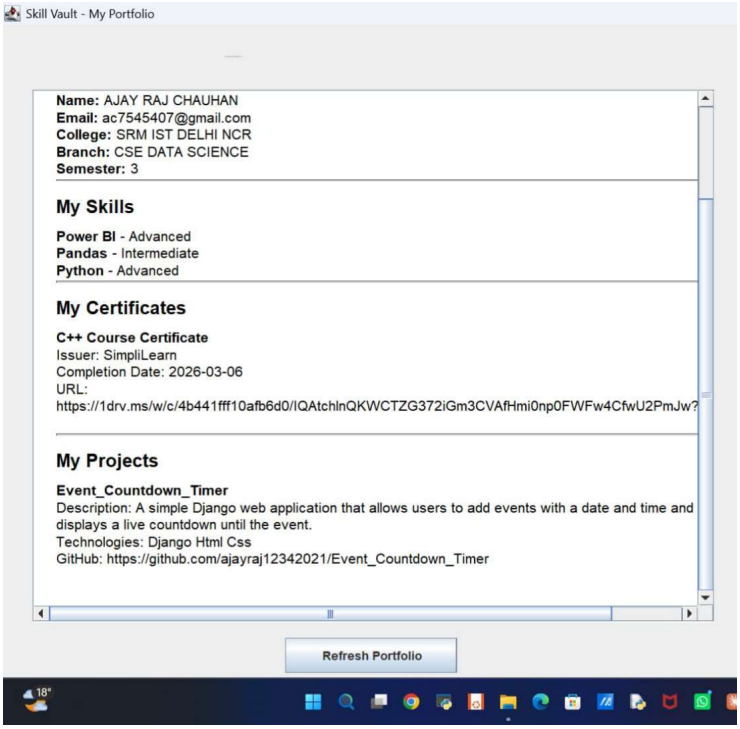


Figure 6.9: Portfolio showing project details

6.7 Software Testing

The test cases below are derived from the implemented features and expected behaviour. The screenshots provide evidence of the screens and of the saved-list and portfolio displays. Tests that depend on database state or on negative inputs should be executed against the configured MySQL server before being marked as passed.

Table 6.1: Application test cases

ID	Function	Test input / action	Expected output	Status
TC-01	Registration	Enter valid student details.	Account is created and success feedback appears.	To be verified
TC-02	Login	Use valid registered credentials.	Dashboard opens for the account.	Shown in screenshots; verify database
TC-03	Login	Enter invalid credentials.	Login error message appears.	To be verified
TC-04	Skills	Add a skill and proficiency level.	Skill appears in the saved skills list.	Shown in screenshots

ID	Function	Test input / action	Expected output	Status
TC-05	Skills	Select and update or delete a skill.	Selected record is changed or removed.	To be verified
TC-06	Projects	Enter title, description, technologies and URL.	Project is saved and appears in the list.	Shown in screenshots
TC-07	Certificates	Enter certificate details.	Certificate appears in the certificate list.	Shown in screenshots
TC-08	Portfolio	Open or refresh My Portfolio.	Profile, skills, certificates and projects are displayed.	Shown in screenshots

6.8 Limitations and Known Issues

The results show that the main screens, the skill, project and certificate views, and the portfolio are implemented. The following limitations were identified from the screenshots and the review of the report.

Table 6.2: Known limitations and planned fixes

Limitation	Planned improvement
Database credentials may be hard-coded.	Move credentials to environment-based configuration; use least-privilege database users.
Password storage and verification need hardening.	Store passwords using a salted, adaptive hashing algorithm.
Limited input validation (email, date, URL, duplicates).	Add format checks and clearer confirmation dialogs.
Certificates can be viewed but there is no update action.	Add an update option consistent with the skills and projects modules.
Several test cases are not yet verified.	Execute TC-01, TC-03 and TC-05 against the live database and record the results.

7. CONCLUSION

7.1 Conclusion

Skill Vault provides a centralised desktop application for organising student profile information and achievements. Its modules support student registration and login, skill tracking, project management, certificate records and a consolidated portfolio. The Java Swing interface, DAO-based data access, JDBC connectivity and MySQL schema demonstrate the practical use of object-oriented programming and database concepts. The screenshots show the core user-facing screens and the portfolio presentation.

7.2 Future Scope

- Add secure password hashing and remove hard-coded database credentials.
- Add email-format and certificate-date validation, plus clearer confirmation dialogs.
- Allow the portfolio to be exported to PDF and shared as a read-only link.
- Add search and filtering for skills, projects and certificates.
- Improve interface responsiveness and visual consistency across screen sizes.
- Add role-based access and automated database backup options.

8. REFERENCES

[1] Project source archive: SKILL-VAULT(FINAL).zip, including the Java source files and database.sql.

[2] Oracle. The Java Tutorials. https://docs.oracle.com/javase/tutorial/

[3] Oracle. Creating a GUI with Swing (Java Swing Tutorial). https://docs.oracle.com/javase/tutorial/uiswing/

[4] Oracle. JDBC Basics. https://docs.oracle.com/javase/tutorial/jdbc/

[5] MySQL. MySQL Reference Manual. https://dev.mysql.com/doc/

[6] MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/